Privacy-Preserving & Trustworthy Cyber-Infrastructures

CIS 4212/6214 (Spring 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Saif E. Nouma***

Homework 6 (Due: April 19th 2026 - 23:59)[100 Points]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

0.1 Programming Assignment – Tree-based Group Diffie-Hellman (TGDH) [90 points]

This assignment will have you implement a Tree-based Group Diffie-Hellman (TGDH) key agreement protocol. TGDH extends the standard two-party DH protocol to a group setting using a binary key tree. Recall how standard Diffie-Hellman establishes a shared secret between two parties using modular exponentiation. TGDH generalizes this by arranging group members as leaves of a binary tree, where each internal node's key is computed by combining the keys of its children via DH-like operations. The root key serves as the group secret shared by all members.

Protocol Description

DH Parameters (RFC 2409, Group 2):

- Prime p: 1024-bit MODP prime (provided in **params_p.txt**)
- Generator $g = 2$ (provided in **params_g.txt**)

Key Tree Structure:
Group members $m_0, m_1, \ldots, m_{n-1}$ are leaves of a balanced binary tree built using **left-heavy splitting**: for n members, the left subtree contains $\lceil n/2 \rceil$ members and the right subtree contains $\lfloor n/2 \rfloor$ members. This splitting is applied recursively. Each node v in the tree has:

- A **secret key** K_v
- A **blinded key** $BK_v = g^{K_v} \bmod p$

Key Computation:
Leaf nodes:
1) $K_{leaf} = H(\text{seed})$, where H is SHA-256
2) $BK_{leaf} = g^{K_{leaf}} \bmod p$

Internal nodes:
1) $K_{parent} = BK_{left}^{K_{right}} \bmod p = g^{K_{left} \cdot K_{right}} \bmod p$
2) $BK_{parent} = g^{K_{parent}} \bmod p$

Group key:
$K_{group} = K_{root}$

The group key is computed bottom-up: leaf keys are derived from seeds, then each internal node's key is computed from its children's keys, propagating up to the root.

Implementation Setting

- Use the OpenSSL BIGNUM library (`openssl/bn.h`) for all modular arithmetic operations. The key function is `BN_mod_exp()` for computing $g^K \bmod p$ and $BK^K \bmod p$.
- Use OpenSSL SHA-256 (`openssl/sha.h`) to derive member secrets from seed strings: $K_{leaf} = \text{SHA256}(\text{seed})$.
- All file I/O uses hex-encoded strings (uppercase).
- **We have provided a template that contains all the necessary functions and their description so that you could use them to create your solution ("RequiredFunctionsTGDH.c").**
- **Also, feel free to use any function of your choice as long as it produces the correct output. Refer to the test vectors to understand the grading criteria.**

Part 1: Tree Setup [25 pt]

$K_{group} \leftarrow TGDH.Setup(p, g, seed_0, seed_1, \ldots, seed_{n-1})$

1) $\forall\, i \in \{0, \ldots, n-1\} : K_{m_i} = \text{SHA256}(seed_i)$
2) $\forall\, i \in \{0, \ldots, n-1\} : BK_{m_i} = g^{K_{m_i}} \bmod p$
3) Build balanced binary tree (left-heavy split)
4) Compute internal keys bottom-up: $K_{parent} = BK_{left}^{K_{right}} \bmod p$
5) $K_{group} = K_{root}$

Your implementation (setup.c) performs the following:

1. Read DH parameters p and g from files.
2. Read member seed files (one per member). Derive each member's secret key via SHA-256.
3. Build the TGDH key tree using left-heavy balanced splitting.
4. Compute all internal node keys bottom-up using $K_{parent} = BK_{left}^{K_{right}} \bmod p$.
5. Write the group key (hex) to **"group_key_setup.txt"**.
6. Write all blinded keys (hex, one per line) to **"blinded_keys_setup.txt"**:
   - First: leaf blinded keys, left to right
   - Then: internal node blinded keys in post-order (children before parent)

Terminal
Compile with the following command:
gcc setup.c -lcrypto -o Setup
Run your code with the following command:
./Setup params_p.txt params_g.txt setup1_seed0.txt setup1_seed1.txt setup1_seed2.txt setup1_seed3.txt

NOTE: In this homework, we will consider $n = 4$.

Part 2: Member Join [20 pt]

When a new member joins, the **sponsor** (rightmost existing member) generates a new secret. The tree is rebuilt with $n + 1$ members.

$K'_{group} \leftarrow TGDH.Join(p, g, \text{existing_secrets}, \text{new_secret}, \text{sponsor_new_secret})$

1) Sponsor m_{n-1} replaces secret with sponsor_new_secret
2) New member m_n is appended with new_secret
3) Rebuild tree with $n + 1$ members using left-heavy split
4) $K'_{group} = K'_{root}$

Your implementation (join.c) performs the following:

1. Read DH parameters.
2. Read existing member secrets from a multi-line file (hex, one secret per line).
3. Read the new member's secret and the sponsor's updated secret from files (hex).
4. Replace the last existing member's secret (the sponsor) with the sponsor's new secret.
5. Append the new member's secret.
6. Rebuild the tree and compute the new group key.
7. Write **"group_key_join.txt"** and **"blinded_keys_join.txt"**.

Terminal
gcc join.c -lcrypto -o Join
./Join params_p.txt params_g.txt join1_existing_secrets.txt join1_new_secret.txt join1_sponsor_new_secret.txt

Part 3: Member Leave [20 pt]

When a member leaves, the **sponsor** (sibling of the departing member) generates a new secret. The tree is rebuilt with $n-1$ members.

For a tree built with left-heavy splitting over n members, member m_i's sibling is determined by the tree structure. In the provided test vectors, the sponsor for a leaving member at an even index is the next member (m_{i+1}), and for an odd index, it is the previous member (m_{i-1}).

$K'_{group} \leftarrow TGDH.Leave(p, g, \text{secrets}, \text{leaving_idx}, \text{sponsor_new_secret})$

1) Remove $m_{\text{leaving_idx}}$ from the member list
2) Sponsor (sibling of leaving member) replaces secret
3) Rebuild tree with $n-1$ members
4) $K'_{group} = K'_{root}$

Your implementation (leave.c) performs the following:

1. Read DH parameters.
2. Read all current member secrets (hex, one per line).
3. Read the index of the leaving member and the sponsor's new secret.
4. Determine the sponsor (sibling of the leaving member).
5. Remove the leaving member, update the sponsor's secret.
6. Rebuild the tree with $n-1$ members and compute the new group key.
7. Write **"group_key_leave.txt"** and **"blinded_keys_leave.txt"**.

Terminal

```
gcc leave.c -lcrypto -o Leave
./Leave params_p.txt params_g.txt leave1_member_secrets.txt leave1_leaving_index.txt
    leave1_sponsor_new_secret.txt
```

Part 4: Tree Merge [15 pt]

Two existing groups merge into one. All members from both groups are combined and a new tree is constructed.

$K'_{group} \leftarrow TGDH.Merge(p, g, \text{group1_secrets}, \text{group2_secrets})$

1) Concatenate group 1 and group 2 member secrets
2) Build new tree with all $n_1 + n_2$ members (group 1 first)
3) $K'_{group} = K'_{root}$

Your implementation (merge.c) performs the following:

1. Read DH parameters.
2. Read Group 1 secrets and Group 2 secrets from separate files.

3. Combine: Group 1 members first, then Group 2 members.
4. Build the merged tree and compute the new group key.
5. Write **"group_key_merge.txt"** and **"blinded_keys_merge.txt"**.

Terminal
gcc merge.c -lcrypto -o Merge
./Merge params_p.txt params_g.txt merge1_group1_secrets.txt merge1_group2_secrets.txt

Part 5: Key Refresh [20 pt]

A member updates their secret key. The tree is rebuilt with the new secret, causing the group key to change.

$K'_{group} \leftarrow TGDH.Refresh(p, g, \text{secrets}, \text{refresh_idx}, \text{new_secret})$

1) Replace $K_{m_{\text{refresh_idx}}}$ with new_secret
2) Rebuild tree with updated secrets
3) $K'_{group} = K'_{root}$

Your implementation (refresh.c) performs the following:

1. Read DH parameters.
2. Read all current member secrets.
3. Read the index of the refreshing member and their new secret.
4. Replace the specified member's secret.
5. Rebuild the tree and compute the new group key.
6. Write **"group_key_refresh.txt"** and **"blinded_keys_refresh.txt"**.

Terminal
gcc refresh.c -lcrypto -o Refresh
./Refresh params_p.txt params_g.txt refresh1_member_secrets.txt refresh1_member_index.txt refresh1_new_secret.txt

Programming Assignment - Specific Notes:

- Compile all programs with: **gcc [file].c -lcrypto -o [Program]**
- All programs read DH parameters from **params_p.txt** and **params_g.txt**.
- All secrets and keys in files are hex-encoded strings (uppercase).
- The tree must use **left-heavy balanced splitting**: left_n = $\lceil n/2 \rceil$, right_n = $\lfloor n/2 \rfloor$.
- Internal node keys are computed as $K_{parent} = BK_{left}^{K_{right}} \bmod p$.
- Each blinded keys file must list leaf BKs (left to right) first, followed by internal BKs in post-order.
- For your convenience, we have provided template files with necessary functions and descriptions: **RequiredFunctionsTGDH.c**
- In the template files, you are free to modify or write your own functions, just ensure you provided the updated template file along with your code submission.
- You can refer to the OpenSSL library documentation `https://www.openssl.org/docs/manmaster/man3/` to learn how to use the functions.
- We have provided a script (VerifyingYourSolutionTGDH.sh) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put your codes, the rest of the provided files along with the "VerifyingYourSolutionTGDH.sh" in one folder and run the following command in the terminal: **bash VerifyingYourSolutionTGDH.sh**
- Ensure the output of your code is correctly named as per the instructions.
- You may look at the testing script to understand how your code is being tested. This script is the same one used for grading on our side, however the test vectors will be different.
- **You Must submit 5 files for this assignment:** **setup.c , join.c , leave.c , merge.c & refresh.c**

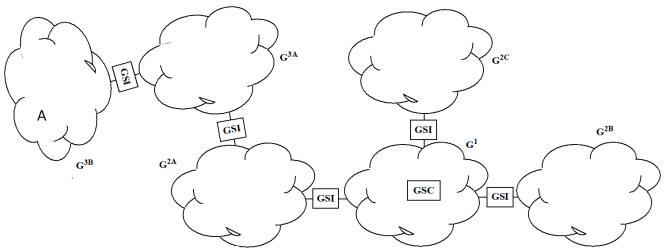
Rubric:

- Part 1 – Setup: Correct **"group_key_setup.txt"** and **"blinded_keys_setup.txt"** for both test sets **[20 pts]**
- Part 2 – Join: Correct outputs and group key differs from pre-join key **[20 pt]**
- Part 3 – Leave: Correct outputs and group key differs from pre-leave key **[20 pt]**
- Part 4 – Merge: Correct outputs and merged key differs from both individual group keys **[15 pt]**
- Part 5 – Refresh: Correct outputs and group key differs from pre-refresh key **[15 pt]**

0.2 Analysis Questions [10 points]

1. Iolus Key Distribution Analysis.

Consider the following network configuration that uses Iolus for group key management:
When the GSC (Group Security Controller) distributes a new group key to member A, analyze:

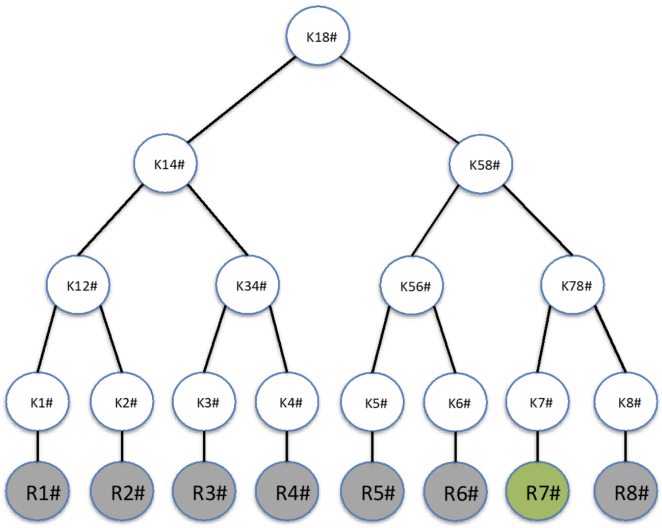


a. How many encryption and decryption operations occur before A receives the new key? **[2.5 pt]**

a. Which entities perform each encryption and decryption operation in the process? Describe the sequence of operations. **[2.5 pt]**

2. LKH Key Update Analysis.

Consider the following LKH (Logical Key Hierarchy) tree structure:



When member R7 is removed from the group:

a. Identify all the keys that must be changed to maintain forward secrecy. Explain why each key needs to be updated **[2.5 points]**

a. Describe one efficient way to distribute the new keys to the remaining group members. Include the sequence of key encryptions and which members receive each encrypted key. **[2.5 pt]**

0.3 Extra Credit: On TGDH Security and Efficiency [20 points]

Consider a TGDH group of n members. Analyze the following:

1. What is the computational cost (in terms of modular exponentiations) for a single member to compute the group key after a Join event? Express your answer in terms of the tree height h.
2. In standard (two party) Diffie-Hellman, the Decisional Diffie-Hellman (DDH) assumption ensures that g^{ab} is computationally indistinguishable from a random group element given g^a and g^b. Discuss how the security of TGDH relies on the DDH assumption at each level of the tree. Is a single DDH assumption sufficient, or does TGDH require a stronger assumption?
3. Compare the communication complexity of TGDH (number of messages and blinded keys broadcast) for Join, Leave, and Merge events against a naive approach where all n members perform pairwise Diffie-Hellman key exchanges.

Justify your answers with the appropriate mathematical and algorithmic rigor.

Write your solution to **hw6.pdf**

Common Assignment Notes:

- Overall you should be submitting 6 files total:
  - **setup.c, join.c, leave.c, merge.c, refresh.c** for the Programming Assignment
  - **hw6.pdf** for Extra Credit
- For your convenience, we have provided template files with necessary functions and descriptions:
  - **RequiredFunctionsTGDH.c** for the Programming Assignment. *Assume it exists on the TA's machine.* DO NOT CHANGE THIS FILE!
- You can refer to the OpenSSL library documentation `https://www.openssl.org/docs/manmaster/man3/` to learn how to use the functions.
- We have provided test files and verification scripts.
- In grading, different seeds/secrets will be used and everything must work.
- Code that throws errors or does not compile/run on our side receives zero credit.
- Follow all specific file names mentioned above, otherwise there will be 20% deduction for inconvenience.
- Please note that version suffixes automatically appended by Canvas to file names are not considered violations.
- Submit your code only if it is functional and passes the provided test cases.